

Redaction Before Delegation: Benchmarking Local PII Filters on Consumer-GPU-Class Hardware

Sveinn Pálsson

May 2026

Abstract

Cloud-hosted language models and retrieval systems are increasingly used with private text from documents, notes, logs, issue trackers, and local knowledge stores. A practical mitigation is to redact personally identifiable information (PII) locally before text crosses an application or model-provider trust boundary. We benchmark local PII and sensitive-span filtering architectures on consumer-GPU-class hardware, comparing deterministic rules, Presidio, OpenAI Privacy Filter (OPF), OPF plus deterministic safety rules, and local decoder LLM redaction. On an AI4Privacy-derived validation subset with 1,065 cases and 4,201 expected sensitive spans, OpenAI Privacy Filter leaves 444 visible spans at roughly 20 ms per case; adding deterministic safety rules reduces visible leakage to 395 spans with additional over-redaction; Qwen3.5-27B direct local redaction leaves 284 visible spans but requires roughly 3.7 s per case. On a synthetic 200-case local-workspace stress set, OPF plus deterministic safety rules substantially improves coverage of file paths, handles, account-like identifiers, URLs, and secret-like strings, while the local decoder remains the highest-capacity but slowest comparator. The results suggest that specialized local token-classification redactors are the strongest low-latency baseline for standard PII redaction. Local decoder LLMs reduce residual leakage in some settings and offer prompt-level policy flexibility, but at substantially higher latency and memory cost. The benchmark therefore reports visible leakage, wrong-label hides, over-redaction, latency, peak GPU memory, taxonomy mismatch, uncertainty, and reproducibility metadata alongside aggregate F1.

1 Introduction

Large language model applications increasingly process text that users would prefer not to disclose in raw form: personal documents, email and chat snippets, developer logs, issue comments, OCR extractions, and retrieved context from local knowledge stores. In many practical systems, some portion of that text is delegated to an external SaaS system such as a hosted model, hosted vector database, or search/indexing service that returns document chunks to the application. Users may want the capability of these services without exposing raw private content to infrastructure outside their control.

We use personally identifiable information (PII) in an operational redaction sense: spans whose disclosure may identify a person, reveal contact or account information, or expose sensitive values such as handles, local file paths, URLs, and secret-like strings. This usage is broader than some legal definitions of PII; here, PII is a redaction target rather than a compliance category.

This paper studies *redaction before delegation*. The redactor runs locally, before text is sent to a hosted model or retrieval service. A local redaction layer can reduce exposed sensitive spans, but it can still miss values, assign the wrong placeholder category, or remove useful context. We therefore evaluate how much sensitive text remains visible, how much non-sensitive text is hidden, and what the systems cost to run.

Recent specialized local PII detectors change the relevant baseline. OpenAI Privacy Filter (OPF) is a local token-classification redactor designed for high-throughput PII detection and masking [1]. It labels tokens in a single pass and decodes spans, rather than generating output autoregressively. A generative local LLM provides a different kind of flexibility. It can be prompted for task-specific policies, unsupported categories, ambiguous cases, structured span lists, or replacement dictionaries. That flexibility comes with higher latency, higher memory use, and output-validation requirements. We compare these methods as local redaction systems with measurable leakage, utility, and runtime behavior.

Privacy filtering is most useful when it can run before text leaves the user’s machine. We therefore use a single NVIDIA RTX 3090 GPU with 24 GB of VRAM as the test hardware [2]. This hardware makes the tradeoff concrete: regex is cheap and reproducible but brittle; Presidio provides a mature local recognizer SDK baseline [3]; OpenAI Privacy Filter provides a specialized local token-classification baseline [1, 4]; generative local LLMs may capture contextual identifiers but add latency, require more memory, and require output parsing.

The contribution is a reproducible local-redaction benchmark under a consumer-GPU-class constraint. The benchmark compares deterministic rules, Presidio, OPF, OPF with deterministic safety rules, and a quantized local decoder using both label-aware and leakage-oriented metrics. It also adds a synthetic local-workspace stress set covering personal notes, chat-like text, file paths, developer outputs, configuration snippets, URLs, account-like identifiers, and OCR-like noise. Results are reported with visible leaks, wrong-label hides, over-redaction, latency, peak GPU memory, observed system failures, and reproducibility metadata.

2 Problem Setting

The trusted environment consists of the user’s machine, storage, redaction process, and any model runtime executed on the machine. An agent wrapper or local knowledge-store interface may coordinate retrieval and redaction, but raw text is expected to stay inside this environment until redaction has run. Hosted services, including LLM providers, vector databases, and search/indexing services, are treated as external.

The primary goal is to reduce sensitive-span exposure before text is sent outside the trusted environment. The primary failure of concern is a visible leak: an expected sensitive value remains visible in the redacted output. Secondary failures are wrong-label hides and over-redaction. System failures such as unusable output or timeout matter operationally because they can leave sensitive text unprocessed, but they are not separate privacy categories in the benchmark.

The non-goals are equally important. This benchmark does not evaluate legal anonymization, HIPAA de-identification, GDPR compliance, or protection against further inference. NIST frames PII protection as context-dependent protection against inappropriate access, use, and disclosure [5]; HHS guidance distinguishes formal HIPAA de-identification routes such as Expert Determination and Safe Harbor [6].

3 Related Work

3.1 PII Protection and De-identification

PII redaction is related to, but distinct from, formal anonymization and regulated de-identification. NIST emphasizes context when identifying and protecting PII [5]. HHS de-identification guidance describes formal healthcare-specific methods and residual risk rather than a general guarantee from

surface masking alone [6]. We adopt that conservative framing: local redaction can reduce exposure, but it does not certify that the remaining text is anonymous or compliant.

3.2 Classical PII and PHI Redaction

Clinical de-identification systems have long combined dictionaries, regular expressions, and heuristics. Neamatullah et al. developed a free-text medical-record de-identification system using such methods [7]. The i2b2/UTHealth shared task provided a widely used clinical de-identification benchmark over longitudinal narratives with surrogate PHI [8]. Philter further illustrates the value of customizable open-source filtering for clinical notes [9]. Presidio is a mature local recognizer SDK baseline most relevant to this benchmark because it combines named entity recognition (NER), rules, regexes, deny-lists, context, and custom recognizers [3].

3.3 Benchmarks and Datasets

AI4Privacy’s PII-Masking-300k dataset is designed for training and evaluating systems that remove PII from text, especially in AI-assistant and LLM contexts [10]. The Text Anonymization Benchmark (TAB) broadens evaluation beyond ordinary named-entity tagging by including privacy/utility-oriented annotations over legal text [11]. Our benchmark uses an AI4Privacy-derived validation subset for controlled evaluation and adds a synthetic local-workspace stress set to expose formats common in private retrieval workflows: paths, handles, logs, configuration fragments, issue comments, URLs, account-like identifiers, and OCR-like noise.

3.4 LLM-era Privacy Filters and Delegation

Recent work studies LLMs as privacy risks, redactors, and delegation components. PAPILLON evaluates privacy-conscious delegation using trusted local/weaker models and untrusted stronger models [12]. PRvL evaluates LLM architectures for PII redaction while measuring redaction performance, semantic preservation, leakage, latency, and computational cost [13]. LegalGuardian studies legal-domain masking and unmasking with NER and local LLMs [14]. LLM-Redactor evaluates several privacy-preserving request techniques, including local-only inference and redaction with placeholder restoration [15]. OpenAI Privacy Filter is directly relevant because it is an open-weight local PII detection/redaction model intended for high-throughput masking workflows [1]. Its model card describes a bidirectional token classifier with span decoding over eight privacy categories and emphasizes that it is not an anonymization or compliance guarantee, that its label policy is static, and that in-domain evaluation and calibration remain necessary for deployment [4].

3.5 Local Inference Constraints

Local redaction depends on efficient local inference. Quantization methods such as AWQ and GPTQ reduce memory requirements for local LLM deployment [16, 17]. The practical GGUF and llama.cpp ecosystem makes quantized local inference accessible outside datacenter settings [18]. These constraints motivate reporting model identity, quantization, runtime, context length, latency, system failures, and peak memory where available.

4 Redaction Architectures

The evaluated conditions cover four local architecture classes. The rule-based baseline uses deterministic regular expressions for common formats and high-confidence patterns. Presidio default

represents a mature recognizer system with NER, rules, and context; the Presidio custom row adds local pattern recognizers for handles, paths, account-like IDs, tickets, secrets, JWTs, private-key headers, and cloud resource IDs. OpenAI Privacy Filter represents the specialized token-classification family: it predicts privacy labels over tokens and decodes spans under an eight-label policy. The decoder LLM row uses a local quantized GGUF runtime, a concise redaction prompt, temperature 0, and a redacted-text output contract.

This distinction drives the comparison. A specialized token classifier gives a fast fixed-label redaction baseline, while a decoder LLM provides prompt-level flexibility for task-specific policies, unsupported categories, and ambiguous cases. A decoder can be asked to rewrite text, emit spans, return a redaction dictionary, or follow a structured output schema, but those choices introduce additional validation and parsing requirements. OPF is also adaptable by other means: its model card describes runtime decoding controls and further fine-tuning for target distributions or local privacy policies [4]. We do not fine-tune OPF or optimize decoder prompts, output schemas, or redaction dictionaries here. The reported rows are fixed, reproducible operating points: direct OPF inference, OPF plus deterministic safety rules, and one direct decoder-redaction configuration.

The matrix includes Presidio custom recognizers, default OPF decoding, an OPF deterministic safety-pass variant, and a Qwen3.5-27B decoder LLM. This paper reports only direct redaction rows and deterministic safety-pass variants; sequential OPF-to-decoder systems are left to future work.

5 Datasets, Label Policies, and Metrics

The benchmark separates label-aware correctness from leakage-oriented privacy risk. A label-aware true positive requires the expected sensitive value to be hidden and the emitted placeholder category to match the benchmark taxonomy after documented normalization. A binary hidden span requires only that the original value is no longer visible. This distinction matters for privacy risk because visible leakage is the primary failure, while label-aware F1 is a secondary correctness metric.

Label policy is itself an experimental variable. OpenAI Privacy Filter predicts eight categories: private person, private address, private email, private phone, private URL, private date, account number, and secret [1]. Our benchmark sets use a normalized placeholder taxonomy designed for cross-system scoring. The OPF rows are scored under that normalized mapping and under binary hide-vs-visible metrics. We do not separately report an OPF-compatible eight-label score, so the benchmark can conflate privacy-preserving hiding with fine-grained taxonomy mismatch.

The main metrics are defined as follows. Under the exact matcher used here, a *visible leak* occurs when an expected sensitive value remains visible in the output. A *wrong-label hide* occurs when the value is hidden but the placeholder category does not match the expected label. An *over-redaction* is a placeholder or redacted span that cannot be attributed to an expected sensitive span under the benchmark matching policy. We also log parser failures and timeouts as system failures, because a caller needs to know whether a redactor returned usable output for every case.

5.1 Matching and Normalization

The scorer evaluates the final redacted string directly. For each annotated sensitive span, it searches the redacted output for the exact original value. If the value is still present, the span is counted as visibly leaked; if the value is absent, it is counted as hidden. The scorer does not apply case folding, Unicode normalization, whitespace cleanup, punctuation stripping, stemming, fuzzy matching, or semantic matching.

Hidden spans are then checked against the emitted placeholder category when one can be aligned to that span. A hidden span with the wrong placeholder category is counted as a wrong-label

hide. Placeholder labels are parsed from `<REDACTED_...>` tokens, normalized by uppercasing and removing numeric or one-letter suffixes, and mapped into the benchmark taxonomy. Duplicate expected spans are counted as separate annotation instances. Extra placeholders that cannot be matched to expected annotations are counted as case-level over-redaction. This exact-match policy is auditable, but it can miss paraphrased disclosures, partial leaks, or generated sensitive-looking values that do not exactly reproduce an annotated source value.

We estimate uncertainty with case-level nonparametric bootstrap intervals. For each system and split, we resample cases with replacement 10,000 times and recompute aggregate precision, recall, F1, F2, hide rate, visible leaks per 1,000 expected spans, wrong-label hides, over-redactions, and latency quantiles. Resampling cases rather than individual spans preserves dependence among sensitive spans within the same example. The intervals describe sampling variability within each evaluated benchmark split.

5.2 Comparison to OPF Model-Card Metrics

Our OPF results should not be read as a direct reproduction of the OpenAI model-card F1 numbers. The model-card evaluation maps PII-Masking-300k categories into OPF’s eight native categories, excludes categories without a corresponding OPF definition, evaluates a holdout under token-level and span-level F1, and reports both baseline and corrected metrics after dataset-label review [4]. In that setting, the model card reports PII-Masking-300k baseline token precision/recall/F1 of 0.940/0.980/0.960 and corrected token precision/recall/F1 of 0.968/0.981/0.974, with lower span-level F1 values of 0.926 and 0.942 respectively [4]. Our main deployment metrics instead use a fixed AI4Privacy-derived validation subset, a normalized cross-system placeholder taxonomy, visible-leak counting, wrong-label hide counting, and over-redaction counting. These metrics intentionally penalize taxonomy mismatch and residual recoverability rather than measuring only token-level OPF-category detection. We therefore report OPF primarily as a deployment baseline under our benchmark policy, not as a reproduction of the model-card score.

The AI4Privacy-derived validation subset contains 1,065 cases and 4,201 expected sensitive spans. A larger subset from the AI4Privacy training partition contains 3,817 cases and 16,148 expected spans; it is used only as an auxiliary consistency check. The synthetic local-workspace stress set contains 200 cases and 656 labeled spans. It is a targeted probe of local-document, developer-log, account-identifier, URL, file-path, handle, and secret-like formats. It is synthetic and contains no real private data.

The validation and auxiliary training-partition subsets are derived from AI4Privacy PII-Masking-300k [10]. Because the license restricts redistribution and derivative sharing [19], the public repository provides reconstruction scripts, scoring code, local runners, aggregate table inputs, model/runtime metadata, and table-rendering scripts, but does not redistribute AI4Privacy-derived text or raw model outputs over that text. Users who wish to reproduce those evaluations must obtain the dataset under its own license terms. The repository is available at <https://github.com/sveinnpalsson/pii-redaction-eval>.

6 Experimental Setup

We run the deterministic rules, Presidio variants, OPF variants, and decoder-LLM on the same three evaluation sets: the AI4Privacy-derived validation subset, the auxiliary AI4Privacy training-partition subset, and the synthetic test set. All model-backed evaluations were run locally on the same RTX 3090 machine. No benchmark input was sent to a hosted model service during evaluation.

The deterministic regex baseline uses fixed Python regular expressions. Presidio default uses the default local recognizers; Presidio custom adds eight local pattern-recognizer groups for handles, paths, account-like IDs, tickets, secrets, private-key headers, and cloud resource IDs. OpenAI Privacy Filter was run through its local Python inference path with default Viterbi span decoding and a deterministic safety-pass variant for high-confidence secrets, paths, handles, contact strings, URLs, and account-like values. The decoder LLM row uses the Qwen/Qwen3.5-27B model [20] through the `lmstudio-community/Qwen3.5-27B-GGUF` quantized version [21], specifically `Qwen3.5-27B-Q4_K_M.gguf`. For reproducibility, our public repository records model identifiers, decoding settings, quantization, prompts, and evaluation-set hashes.

ID	Architecture	Implementation	Evaluation role
R1	Deterministic rules	Regex rules for common sensitive formats	Low-cost rule baseline
P1	Recognizer system	Presidio default local recognizers, including NER, rules, and context	Mature recognizer baseline
P2	Recognizer system + local patterns	Presidio plus patterns for handles, paths, tickets, secrets, cloud IDs, and account-like IDs	Domain-rule comparison
OPF-D	Specialized PII model	OpenAI Privacy Filter with default Viterbi span decoding	Fast specialized-model baseline
OPF+R	Specialized PII model + safety rules	OPF-D plus deterministic rules for high-confidence local identifiers and contact values	Safety-first variant
Q27-D	Generative decoder LLM	Qwen3.5-27B through a local Q4_K_M GGUF runtime	High-capacity decoder LLM

Table 1: Overview of the main local redaction architectures.

7 Results

Table 2 reports the main validation results. Visible leaks are the primary privacy failure. Regex hides only 23.6% of expected spans and leaves 3,208 visible values. Presidio default improves substantially, hiding 59.6% and leaving 1,696 visible values. Presidio custom hides 60.0% and leaves 1,680 visible values, but the extra recognizers mainly trade fewer visible leaks for more over-redaction on the validation subset. OpenAI Privacy Filter is a much stronger specialized local baseline, hiding 89.4% and leaving 444 of 4,201 expected spans visible. OPF plus deterministic safety rules hides 90.6% and leaves 395 visible spans, at the cost of more over-redaction. Among evaluated validation runs, Qwen3.5-27B direct local redaction has the fewest visible leaks, leaving 284 expected spans visible. That reduction comes at roughly 180 times the mean latency of OPF, so it should be interpreted as a high-capacity local decoder option.

System	F1 $\uparrow$	F2 $\uparrow$	Hide $\uparrow$	Visible $\downarrow$	Visible/k $\downarrow$	Wrong $\downarrow$	Over $\downarrow$
R1	0.252	0.185	23.6% [21.6, 25.7]	3208 [3017, 3399]	763.6 [742.6, 784.2]	335	26
P1	0.498	0.449	59.6% [57.3, 61.9]	1696 [1563, 1836]	403.7 [380.7, 426.7]	736	393
P2	0.498	0.457	60.0% [57.7, 62.3]	1680 [1548, 1822]	399.9 [377.1, 422.7]	702	580
OPF-D	0.733	0.727	89.4% [88.1, 90.7]	444 [388, 503]	105.7 [93.2, 118.7]	721	329
OPF+R	0.720	0.727	90.6% [89.4, 91.8]	395 [342, 449]	94.0 [82.4, 105.8]	733	534
Q27-D	0.762	0.773	93.2% [91.6, 94.8]	284 [217, 359]	67.6 [52.2, 84.3]	634	502

Table 2: Results on 1,065 AI4Privacy-derived cases with 4,201 expected spans. Visible leakage is the primary privacy failure. Bootstrap intervals are case-level 95% percentile intervals.

The label-aware tradeoff remains visible. Qwen3.5-27B reduces leakage most strongly, but it also produces 634 wrong-label hides and 502 over-redactions. OpenAI Privacy Filter produces fewer over-redactions than Qwen3.5-27B, but more visible leaks. The deterministic safety pass reduces OPF’s visible leakage while increasing over-redaction from 329 to 534 spans on validation. Presidio custom shows the same pattern at a lower accuracy tier: it reduces visible leaks only modestly on validation, while increasing over-redaction from 393 to 580. A deployment focused only on reducing visible leakage may rank rows differently from a deployment that needs stable placeholder categories for restoration or automation.

Runtime is summarized in Table 3. In our local measurements, deterministic regex rules were effectively free, running below 0.1 ms per case. Presidio default and custom configurations ran at roughly 10 ms per case on CPU, but left substantially more visible spans than OPF. OPF ran at roughly 20 ms per case on CUDA and peaked at about 3.4 GiB of GPU memory. Qwen3.5-27B direct redaction was the highest-capacity comparator, but averaged roughly 3.7 s per case and peaked at about 17.7 GiB. This makes Q27-D the high-capacity, high-cost point in the matrix: it gives stronger leakage reduction, while OPF provides the lower-latency and lower-memory option. We observed no parser failures, timeouts, or errors that returned unredacted text in the reported runs.

System	Runtime	Mean ms $\downarrow$	p50 ms $\downarrow$	p95 ms $\downarrow$	p99 ms $\downarrow$	Peak VRAM
R1	CPU regex	0.0	0.0	0.1	–	–
P1	Presidio CPU	9.7	9.3	13.2	16.1	–
P2	Presidio CPU	9.7	9.3	13.1	15.6	–
OPF-D	OPF CUDA	20.5	18.8	22.5	25.6	3.4 GiB
OPF+R	OPF CUDA	20.2	18.5	22.1	24.4	3.4 GiB
Q27-D	GGUF CUDA	3685.3	3459.0	5429.4	6674.4	17.7 GiB

Table 3: Local runtime for reported validation rows. Peak VRAM is the highest GPU memory usage observed in the recorded runtime telemetry for each GPU-backed condition.

Category-level leakage in Table 4 shows why leakage-oriented reporting matters. The best-row column identifies which reported row produces the fewest visible leaks for each category. Residual leaks are concentrated in address fragments, contextual custom identifiers, account-like values, and short personal identifiers. These are values that are easy to miss if evaluation is reduced to aggregate F1 alone, and they overlap with the failure modes highlighted in OPF’s own documentation: domain-specific identifiers, fragmented spans, mixed-format text, and novel secret formats [4].

Category	Expected	Best row	Visible under best	OPF-D visible	OPF+R visible	Q27-D visible
ADDRESS	777	Q27-D	210	261	260	210
CUSTOM	751	Q27-D	47	83	74	47
ACCOUNT	1470	Q27-D	19	35	25	19
PERSON	394	Q27-D	5	32	32	5
PHONE	293	Q27-D	1	6	4	1
EMAIL	516	P1 / OPF+R	0	27	0	2

Table 4: Category-level validation leakage under the best reported row per category. Category winners are selected only among reported rows.

Table 5 evaluates the systems on the synthetic local-workspace stress set, which targets file paths, handles, account-like identifiers, URLs, and secret-like strings. Presidio custom improves substantially over Presidio default on this split, reducing visible leaks from 259 to 102 through additional pattern recognizers. OPF leaves 77 visible spans, while OPF plus deterministic safety rules reduces this to 35 with only a small increase in over-redaction. Qwen3.5-27B direct redaction has the lowest visible leakage on this split, leaving 7 of 656 labeled spans visible. The stress set therefore separates two strengths: OPF+R covers many local identifier formats with low runtime overhead, while Q27-D gives the lowest leakage when additional latency and memory are acceptable.

System	Hide $\uparrow$	Visible $\downarrow$	Visible/k $\downarrow$	Wrong $\downarrow$	Over $\downarrow$
R1	40.4%	391	596.0	20	0
P1	60.5%	259	394.8	93	7
P2	84.5%	102	155.5	114	81
OPF-D	88.3%	77	117.4	110	33
OPF+R	94.7%	35	53.4	113	41
Q27-D	98.9%	7	10.7	28	2

Table 5: Synthetic local-workspace stress split: 200 cases and 656 expected sensitive spans. The stress set is synthetic and is not a real private deployment corpus.

8 Discussion and Deployment Implications

The main practical difficulty is not ordinary email or phone detection, but identifiers whose sensitivity depends on the document and the application. A handle, ticket number, repository path, account-like string, URL, or local file path may be harmless in one context and revealing in another. Deterministic rules help when those patterns are known in advance, such as internal ticket prefixes, home-directory paths, cloud account IDs, API-key prefixes, or organization-specific handles. They also need negative tests on ordinary text, because broad rules can remove numbers, codes, names, or paths that the intended task needs.

Wrong-label hides and over-redaction affect different parts of a deployment. If an email address is hidden under the wrong placeholder type, the original value is not visible, but restoration, auditing, or routing may still fail. If too much text is hidden, the model may lose useful context. This is why the benchmark reports visible leaks, wrong-label hides, over-redaction, latency, and memory separately instead of reducing the comparison to one aggregate score.

The comparison also separates two adaptation paths. Q27-D can be steered through prompts and may support task-specific redaction policies without changing model weights. OPF has a

different adaptation path: decoding controls and fine-tuning for target distributions or local privacy policies [4]. This paper does not optimize either path. It evaluates fixed OPF rows, OPF plus explicit deterministic rules, and one fixed decoder configuration so the system comparison can be easily reproduced.

The reported numbers should be read as benchmark results. The same leakage, over-redaction, latency, and memory measurements should be repeated on target data before using a redactor in a new setting.

9 Conclusion

Local redaction before delegation can materially reduce sensitive-span exposure, but it remains an engineering control rather than a privacy guarantee. On the evaluated consumer-GPU-class hardware, OPF-D and OPF+R are the fast, low-memory options: OPF-D gives the strongest low-latency baseline for standard PII categories, and OPF+R improves coverage of some identifiers at the cost of additional over-redaction. Q27-D achieves the lowest visible leakage, especially on the stress set, showing the value of promptable local decoders for policy-specific redaction when their latency and memory cost are acceptable. Aggregate F1 is useful, but deployment decisions also require concrete failure and cost measures: what remains visible, what was hidden unnecessarily, whether placeholder categories are reliable, and how much latency and memory the system consumes.

AI-use disclosure. The author used AI-tools for editorial assistance and for help implementing and debugging parts of the evaluation code. The author designed the benchmark, reviewed, tested, and verified the released code, ran the experiments, and is responsible for the reported results, released source code, and final manuscript.

A Auxiliary Training-Partition Check

To check whether the validation result depends on a single AI4Privacy partition, we repeat the same scoring procedure on a larger subset sampled from the training partition. The main comparisons remain based on the validation and stress splits.

System	Hide $\uparrow$	Prec.	Recall	F1 $\uparrow$	F2 $\uparrow$	Visible $\downarrow$	Wrong $\downarrow$	Over $\downarrow$
R1	23.2%	0.607	0.142	0.230	0.167	12406	1454	28
P1	57.9%	0.617	0.403	0.488	0.433	6804	2832	1202
P2	57.9%	0.597	0.414	0.489	0.441	6791	2667	1844
OPF-D	89.7%	0.726	0.710	0.718	0.713	1663	3016	1321
OPF+R	90.0%	0.703	0.713	0.708	0.711	1613	3021	1846
Q27-D	93.5%	0.737	0.766	0.751	0.760	1044	2728	1697

Table 6: Auxiliary check on the AI4Privacy training partition. The same broad ordering appears on this larger subset, which supports the validation-set trend.

References

- [1] OpenAI. OpenAI Privacy Filter. Hugging Face model card, <https://huggingface.co/openai/privacy-filter>, 2026. Accessed 5 May 2026.
- [2] NVIDIA. GeForce RTX 3090 Family. <https://www.nvidia.com/en-us/geforce/graphics-cards/30-series/rtx-3090-3090ti/>, 2026. Accessed 5 May 2026.
- [3] Microsoft. Presidio: Data protection and de-identification SDK. <https://microsoft.github.io/presidio/>, 2026. Accessed 5 May 2026.
- [4] OpenAI. Model card for OpenAI Privacy Filter. Technical report, OpenAI, April 2026. <https://cdn.openai.com/pdf/c66281ed-b638-456a-8ce1-97e9f5264a90/OpenAI-Privacy-Filter-Model-Card.pdf>; accessed 5 May 2026.
- [5] Erika McCallister, Timothy Grance, and Karen Scarfone. Guide to protecting the confidentiality of personally identifiable information (PII). NIST Special Publication 800-122, National Institute of Standards and Technology, 2010. DOI: 10.6028/NIST.SP.800-122.
- [6] U.S. Department of Health and Human Services. Guidance regarding methods for de-identification of protected health information in accordance with the HIPAA privacy rule. <https://www.hhs.gov/hipaa/for-professionals/special-topics/de-identification/index.html>, 2025. Accessed 5 May 2026.
- [7] Ishna Neamatullah, Margaret M. Douglass, Li-wei H. Lehman, Andrew Reisner, Mauricio Villarroel, William J. Long, Peter Szolovits, George B. Moody, Roger G. Mark, and Gari D. Clifford. Automated de-identification of free-text medical records. *BMC Medical Informatics and Decision Making*, 8, Article 32, 2008. DOI: 10.1186/1472-6947-8-32.
- [8] Amber Stubbs, Christopher Kotfila, and Özlem Uzuner. Automated systems for the de-identification of longitudinal clinical narratives: Overview of 2014 i2b2/UTHealth shared task track 1. *Journal of Biomedical Informatics*, 58:S11–S19, 2015. DOI: 10.1016/j.jbi.2015.06.007.
- [9] Beau Norgeot, Kathleen Muenzen, Thomas A. Peterson, Xuancheng Fan, Benjamin S. Glicksberg, Gundolf Schenk, Eugenia Rutenberg, Boris Oskotsky, Marina Sirota, Jinoos Yazdany, Gabriela Schmajuk, Dana Ludwig, Theodore Goldstein, and Atul J. Butte. Protected health information filter (Philter): Accurately and securely de-identifying free-text clinical notes. *npj Digital Medicine*, 3, Article 57, 2020. DOI: 10.1038/s41746-020-0258-y.
- [10] AI4Privacy. PII-Masking-300k. Hugging Face dataset card, <https://huggingface.co/datasets/ai4privacy/pii-masking-300k>, 2024. DOI: 10.57967/hf/1995; accessed 5 May 2026.
- [11] Ildikó Pilán, Pierre Lison, Lilja Øvrelid, Anthi Papadopoulou, David Sánchez, and Montserrat Batet. The text anonymization benchmark (TAB): A dedicated corpus and evaluation framework for text anonymization. *Computational Linguistics*, 48(4):1053–1101, 2022. DOI: 10.1162/coli_a_00458.
- [12] Li Siyan, Vethavikashini Chithrra Raghuram, Omar Khattab, Julia Hirschberg, and Zhou Yu. PAPILLON: Privacy preservation from Internet-based and local language model ensembles. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 3371–3390, Albuquerque, New Mexico, 2025. Association for Computational Linguistics. DOI: 10.18653/v1/2025.naacl-long.173.

- [13] Leon Garza, Anantaa Kotal, Aritran Piplai, Lavanya Elluri, Prajit Das, and Aman Chadha. PRvL: Quantifying the capabilities and risks of large language models for PII redaction. arXiv preprint, <https://arxiv.org/abs/2508.05545>, 2025. arXiv:2508.05545; DOI: 10.48550/arXiv.2508.05545.
- [14] M. Mikail Demir, Hakan T. Otal, and M. Abdullah Canbaz. LegalGuardian: A privacy-preserving framework for secure integration of large language models in legal practice. arXiv preprint, <https://arxiv.org/abs/2501.10915>, 2025. arXiv:2501.10915; DOI: 10.48550/arXiv.2501.10915.
- [15] Justice Owusu Agyemang, Jerry John Kponyo, Elliot Amponsah, Godfred Manu Addo Boakye, and Kwame Opuni-Boachie Obour Agyekum. LLM-Redactor: An empirical evaluation of eight techniques for privacy-preserving LLM requests. arXiv preprint, <https://arxiv.org/abs/2604.12064>, 2026. arXiv:2604.12064; DOI: 10.48550/arXiv.2604.12064.
- [16] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. AWQ: Activation-aware weight quantization for on-device LLM compression and acceleration. In *Proceedings of Machine Learning and Systems*, volume 6, 2024. arXiv:2306.00978.
- [17] Elias Frantar, Saleh Ashkboos, Torsten Hoefer, and Dan Alistarh. GPTQ: Accurate post-training quantization for generative pre-trained transformers. arXiv preprint, <https://arxiv.org/abs/2210.17323>, 2022. arXiv:2210.17323; DOI: 10.48550/arXiv.2210.17323.
- [18] Georgi Gerganov and llama.cpp contributors. llama.cpp: LLM inference in C/C++. GitHub repository, <https://github.com/ggml-org/llama.cpp>, 2026. Accessed 5 May 2026.
- [19] AI4Privacy. PII-Masking-300k license. <https://huggingface.co/datasets/ai4privacy/pii-masking-300k/blob/main/LICENSE.md>, 2024. Accessed 5 May 2026.
- [20] Qwen Team. Qwen3.5-72B. Hugging Face model card, <https://huggingface.co/Qwen/Qwen3.5-72B>, 2026. Accessed 5 May 2026.
- [21] LM Studio Community. Qwen3.5-72B-GGUF. Hugging Face model repository, <https://huggingface.co/lmstudio-community/Qwen3.5-72B-GGUF>, 2026. Qwen3.5-72B-Q4_K_M.gguf; SHA256 730aa508f1e48bb680917c16069176e07df86a29dc0c6602fc252c8f93062da9; accessed 6 May 2026.